

* 전체 코드

[illegible]

```

55     temp[cnt] = input[yy * width + xx];
56     cnt++;
57 }
58 }
59 }
60
61 // sorting
62 heap_sort(temp, cnt);
63
64 // median
65 med = temp[cnt / 2];
66
67 output[y * width + x] = med;
68 }
69 }
70
71 free(temp);
72 }
73
74 void downheap(float* a, int n, int k)
75 {
76     int j;
77     float v = a[k];
78
79     while (k < n / 2) {
80         j = 2 * k + 1;
81
82         if (j < n - 1 && a[j] < a[j + 1]) {
83             j++;
84         }
85
86         if (v >= a[j]) {
87             break;
88         }
89
90         a[k] = a[j];
91         k = j;
92     }
93
94     a[k] = v;
95 }
96
97 void heap_sort(float* a, int n)
98 {
99     int i;
100     float t;
101
102     for (i = n / 2 - 1; i >= 0; i--) {
103         downheap(a, n, i);
104     }
105
106     while (n > 1) {
107         t = a[0];
108         a[0] = a[n - 1];
109         a[n - 1] = t;

```

```

109         a[n - 1] = t;
110
111         n--;
112         downheap(a, n, 0);
113     }
114 }
115
116 int main() {
117     int height = 44, width = 45;
118     int window_size = 3;
119
120     float* input = (float*)malloc(sizeof(float) * height * width);
121     float* output = (float*)malloc(sizeof(float) * height * width);
122
123     if (input == NULL || output == NULL) {
124         printf("Memory allocation failed!\n");
125         return -1;
126     }
127
128     char cwd[1024];
129     _getcwd(cwd, sizeof(cwd));
130     printf("Current working directory: %s\n", cwd);
131
132     read_file(input, height, width, "noisy_data.txt");
133     printf("Data loaded successfully.\n");
134
135     median_filtering(input, output, height, width, window_size);
136     printf("Filtering completed.\n");
137
138     FILE* out_fp = fopen("filtered_result2.txt", "w");
139
140     if (out_fp == NULL) {
141         printf("Output file open failed!\n");
142         exit(1);
143     }
144
145     for (int y = 0; y < height; y++) {
146         for (int x = 0; x < width; x++) {
147             fprintf(out_fp, "%.0f\t", output[y * width + x]);
148         }
149         fprintf(out_fp, "\n");
150     }
151
152     fclose(out_fp);
153
154     printf("Filtered result saved to filtered_result.txt\n");
155
156     free(input);
157     free(output);
158
159     return 0;
160 }

```

* 실행 결과

-before(noisy_data.txt)

-b	0	2	0	2	12	0	54	0	0	0	18	0	0	2	42	25	0	0	0	13	0	14	49	0	0	52	0	0	0	1	22	13	0	19	0	0	0	0	0	2	0	0	8
0	0	31	0	0	0	19	1	0	0	0	23	48	0	0	27	0	0	8	0	4	9	14	0	0	0	0	20	8	4	0	0	1	18	49	0	0	0	0	0	22	40	0	14
23	0	0	31	0	34	4	0	16	51	0	0	3	0	60	0	0	17	0	11	0	41	0	0	42	10	57	12	0	0	0	17	0	24	0	0	74	0	1	24	15	8		
52	0	0	0	0	19	3	0	6	44	0	0	42	0	35	0	24	0	0	12	3	0	0	0	0	23	0	0	34	0	0	0	0	0	0	0	0	7	0	2	28	26		
9	8	4	3	0	9	9	0	0	33	0	0	0	0	0	0	0	5	22	0	45	25	0	0	17	12	25	0	0	0	56	31	13	8	16	6	0	36	0	0				
9	0	0	0	24	0	22	18	0	0	0	0	16	18	0	0	14	0	22	0	0	0	233	254	236	255	22	0	11	0	0	16	0	0	4	0	31	0	11	24	0			
8	0	0	52	6	0	0	0	10	0	0	53	0	0	28	49	0	30	10	0	0	255	285	240	255	240	215	217	10	22	30	5	6	21	52	5	30	41	4	8	0	20		
0	0	0	0	0	16	25	0	6	29	28	9	0	0	0	26	13	43	0	16	20	195	222	219	255	255	192	250	221	213	0	0	24	22	0	0	18	16	0	3	12	15		
15	0	0	9	0	1	12	11	0	15	15	48	0	19	0	0	55	9	0	210	234	254	249	249	247	255	237	255	255	8	3	14	21	0	0	3	0	0	11	0	16			
15	0	0	25	50	12	0	0	5	0	0	25	22	7	18	5	33	0	23	255	203	255	223	233	255	0	226	255	255	255	0	10	0	24	7	0	0	3	15	0	16	45	0	
0	0	0	0	0	33	0	29	0	25	16	21	40	13	6	0	12	34	6	255	244	239	238	255	220	35	255	255	255	231	30	51	0	46	0	21	0	12	59	0	0			
8	1	16	6	0	15	24	0	5	11	0	9	0	8	57	0	0	19	0	199	224	226	247	0	8	3	0	211	255	246	255	0	0	7	5	0	33	0	0	0	40	0		
23	2	13	0	0	0	10	0	0	2	0	0	15	7	12	1	0	0	0	248	247	255	239	0	13	1	0	255	246	234	253	253	25	0	0	0	0	0	0	43	0	1		
0	21	30	0	0	56	2	35	9	36	0	4	20	0	23	28	0	0	255	255	243	255	0	0	41	62	37	255	237	230	252	0	0	61	4	0	0	33	28	40	8	0		
0	42	26	0	0	0	31	0	0	0	36	1	15	14	39	23	0	255	255	215	230	1	0	5	0	11	244	255	255	241	0	39	43	22	0	0	0	3	0	0	0			
32	39	0	22	29	0	0	41	0	6	0	10	9	3	0	29	255	255	254	216	255	255	0	0	0	0	9	238	255	255	255	255	33	9	8	42	0	10	0	23	6	44		
0	0	0	0	0	14	0	0	23	5	0	0	8	16	46	0	238	255	255	233	255	0	0	21	0	3	4	12	0	255	255	226	255	0	43	0	20	0	42	0	5	23		
16	17	0	5	15	36	0	0	0	17	14	48	255	255	255	232	255	235	28	26	31	0	0	11	5	0	242	250	255	248	255	250	255	255	14	0	12	24	37	15	27	0		
23	0	0	10	0	0	24	9	0	35	24	0	0	6	24	0	255	234	236	237	43	35	0	19	33	0	33	0	3	255	132	221	255	0	0	6	27	4	14	0	71	0		
11	0	0	0	0	24	9	0	0	17	0	0	7	1	0	255	255	255	255	255	0	0	0	18	17	38	10	0	28	210	255	255	243	199	0	34	0	0	23	0	3	8		
0	0	14	0	0	0	0	47	38	21	0	0	11	24	235	241	238	255	0	16	0	0	38	0	62	16	0	0	255	210	255	255	0	0	0	12	0	0	0	7	18	0		
6	4	0	8	55	9	0	0	5	3	9	0	3	15	0	248	255	255	216	16	0	17	0	10	27	2	11	4	1	255	210	255	255	0	32	0	7	0	19	0	0	56	44	
0	0	15	0	6	40	18	0	6	4	0	0	1	0	226	255	255	240	255	17	18	0	17	35	7	14	0	13	0	238	255	255	247	225	0	17	0	11	10	0	0	0		
39	15	0	0	15	14	0	0	0	17	0	18	0	0	236	233	255	255	0	18	9	18	5	51	8	0	0	64	0	0	193	246	229	255	245	12	0	0	28	0	22	31	29	0
15	17	0	5	15	36	0	0	0	17	14	48	255	255	255	232	255	235	28	26	31	0	0	11	5	0	242	250	255	248	255	250	255	255	14	0	12	24	37	15	27	0		
59	15	0	0	12	24	0	39	4	9	0	52	42	239	248	255	238	248	251	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	
3	0	6	18	4	0	12	0	0	0	0	251	239	253	238	237	231	255	222	233	254	255	249	234	247	255	255	251	255	240	231	226	255	255	247	238	27	0	84	17	15			
0	30	16	0	0	0	26	35	16	0	217	200	255	255	224	255	246	240	255	255	237	242	255	255	255	255	245	245	212	255	253	255	255	243	232	221	0	0	0	0	2	0		
0	13	1	0	0	60	0	0	0	35	255	255	255	245	0	14	15	63	24	251	239	231	245	231	241	255	255	223	1	0	0	228	238	239	255	228	0	11	8	0	39	0		
0	0	34	0	0	6	0	0	5	230	255	255	255	255	29	40	0	29	15	46	2	7	0	4	11	0	0	0	28	0	0	63	244	248	250	198	250	0	16	0	20	13		
19	0	0	0	39	0	13	0	10	230	231	255	255	0	0	3	67	2	4	0	0	11	0	10	0	6	8	10	0	24	18	215	242	255	255	230	255	3	0	14	0			
0	0	50	0	0	0	1	0	229	255	255	255	255	0	0	0	0	17	0	39	17	0	6	16	15	0	0	1	0	1	7	255	255	254	207	242	23	22	15	0	3			
0	47	18	7	0	0	10	250	255	202	221	32	0	0	19	0	0	37	15	17	0	1	16	0	0	36	0	39	2	13	31	14	0	255	255	255	255	42	0	2	23	0		
0	0	24	5	0	2	0	255	255	255	255	0	28	0	0	56	2	24	0	0	1	31	15	0	7	0	0	0	8	11	11	0	226	227	246	255	24	0	8	58	0			
0	0	0	23	48	27	9	0	255	255	236	236	0	0	28	4	0	0	37	0	4	0	64	22	20	1	4	19	0	0	0	17	18	255	255	234	255	0	0	0	14	3		
0	5	0	0	12	0	0	246	255	250	255	255	15	0	6	8	0	0	11	0	0	11	0	0	0	0	27	15	0	21	0	36	245	232	255	255	38	0	0	37	2			
0	0	0	9	48	0	0	255	255	255	255	42	36	7	1	27	0	69	19	0	28	0	8	19	0	9	0	9	0	0	24	18	215	242	255	255	246	0	16	0	38			
0	24	0	2	39	0	2	255	255	255	0	0	50	5	0	84	44	28	0	17	0	10	0	10	0	0	24	49	0	0	21	0	0	20	255	255	255	0	33	0	30	0		
0	0	0	5	0	24	0	41	255	10	0	8	14	0	0	35	2	13	6	0	5	26	0	14	8	32	0	28	0	0	52	30	17	0	24	49	43	27	0	6	18			
16	61	28	0	0	0	28	0	3	13	15	0	13	14	59	18	0	0	0	37	42	14	7	32	21	0	0	0	0	0	0	17	7	0	7	1	16	5	3	14	6	25		
9	0	0	89	54	1	14	30	0	16	27	0	0	0	3	0	28	10	0	24	0	6	0	50	0	0	0	9	0	0	19	0	0	0	32	0	49	7	0	10	19	3		
0	17	0	28	2	0	8	22	3	0	12	27	9	16	0	8	0	0	13	0	0	0	19	0	0	0	18	0	0	0	0	49	0	0	0	34	74	0	22	30	34			
30	12	4	29	0	0	19	0	44	10	0	0	0	31	18	2	0	36	8	0	39	0	29	13	12	0	0	20	7	19	18	13	0	35	14	6	21	51	13	0	36	0		
24	0	0	0																																								

* 코드 분석

1. 코드 동작 원리

이 코드는 noisy_data.txt에 저장된 44×45 크기의 숫자 데이터를 읽어 median filtering을 수행한 뒤, 결과를 새로운 텍스트 파일로 저장하는 프로그램이다. 먼저 main()에서 원본 데이터를 저장할 input 배열과 필터링 결과를 저장할 output 배열을 동적 할당하고, read_file()을 통해 텍스트 파일의 값을 순서대로 읽어 input[y * width + x] 형태로 저장한다. 이후 median_filtering() 함수에서 각 위치 (y, x)를 기준으로 주변 3×3 범위의 값을 temp 배열에 모으고, 이미지 범위를 벗어나는 위치는 제외한다. 모아진 값들은 heap_sort()를 이용해 오름차순으로 정렬되며, 정렬된 값 중 가운데 값인 temp[cnt / 2]를 median 값으로 선택해 output 배열에 저장한다. 모든 위치에 대해 필터링이 끝나면 filtered_result.txt 파일을 생성하고, output 배열의 44×45 결과값을 행과 열 형태로 저장한 뒤 파일을 닫고 동적 할당한 메모리를 해제하면서 프로그램을 종료한다.

2. 코드 구성 요소

```
void read_file(float* input, int height, int width, const char filename[])
{
    int x, y;
    FILE* fp;

    fp = fopen(filename, "r");

    if (fp == NULL) {
        printf("File open failed!\n");
        exit(1);
    }

    for (y = 0; y < height; y++) {
        for (x = 0; x < width; x++) {
            fscanf(fp, "%f", &input[y * width + x]);
        }
        fscanf(fp, "\n");
    }

    fclose(fp);
}
```

이 read_file 함수는 filename으로 전달받은 텍스트 파일을 읽기 모드 "r"로 열고, 파일이 제대로 열리지 않으면 File open failed!를 출력한 뒤 프로그램을 종료한다. 파일이 정상적으로 열리면 height번 행을 반복하고, 각 행마다 width번 열을 반복하면서 fscanf(fp, "%f", &input[y * width + x])로 숫자 하나씩 읽어 1차원 배열 input에 저장한다. 여기서 y * width + x는 2차원 좌표 (y, x)를 1차원 배열 인덱스로 바꾸는 식이라서, 예를 들어 width가 45이면 2번째 줄 3번째 값은 input[1 * 45 + 2]에 저장된다. 한 행을 다 읽은 뒤에는 fscanf(fp, "\n")로 줄바꿈을 넘기고, 모든 데이터를 읽은 후 fclose(fp)로 파일을 닫는다. 즉 이 함수는 noisy_data.txt 같은 44×45 숫자 파일을 통째로 읽어서, 이후 median filtering에 사용할 수 있도록 input 배열에 저장하는 역할을 한다.

```

void median_filtering(float* input, float* output, int height, int width, int window_size)
{
    int x, y, xx, yy;
    int len = window_size / 2;
    int cnt;
    float med;

    float* temp = (float*)calloc(window_size * window_size, sizeof(float));

    if (temp == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    for (y = 0; y < height; y++) {
        for (x = 0; x < width; x++) {
            cnt = 0;

            // data sampling
            for (yy = y - len; yy <= y + len; yy++) {
                for (xx = x - len; xx <= x + len; xx++) {
                    if (yy >= 0 && yy < height && xx >= 0 && xx < width) {
                        temp[cnt] = input[yy * width + xx];
                        cnt++;
                    }
                }
            }

            // sorting
            heap_sort(temp, cnt);

            // median
            med = temp[cnt / 2];

            output[y * width + x] = med;
        }
    }

    free(temp);
}

```

이 `median_filtering` 함수는 `input` 배열에 저장된 원본 데이터를 하나씩 돌면서 각 위치의 주변 값을 모아 median 값으로 바꾼 뒤 `output` 배열에 저장하는 함수이다. 먼저 `window_size / 2`를 이용해 현재 픽셀 기준으로 주변을 얼마나 볼지 정하는데, `window_size`가 3이면 `len`은 1이므로 현재 위치를 중심으로 위·아래·왼쪽·오른쪽을 포함한 최대 3×3 범위를 검사한다. `temp` 배열은 이 주변값들을 임시로 저장하기 위해 동적 할당된 배열이고, 각 픽셀마다 `cnt`를 0으로 초기화한 뒤 범위 안에 있는 값만 `temp[cnt]`에 넣는다. 이때 가장자리 픽셀은 3×3 범위가 이미지 밖으로 나갈 수 있으므로 `yy >= 0`, `yy < height`, `xx >= 0`, `xx < width` 조건을 통해 실제 데이터 범위 안에 있는 값만 사용한다. 주변값을 모두 모은 뒤 `heap_sort(temp, cnt)`로 실제 저장된 `cnt`개의 값만 정렬하고, 정렬된 값 중 가운데 위치인 `temp[cnt / 2]`를 median 값으로 선택한다. 마지막으로 이 median 값을 `output[y * width + x]`에 저장하여 원래 픽셀값을 주변값 기준의 중앙값으로 대체하고, 모든 픽셀 처리가 끝나면 임시 배열 `temp`를 `free`로 해제한다.

```

void downheap(float* a, int n, int k)
{
    int j;
    float v = a[k];

    while (k < n / 2) {
        j = 2 * k + 1;

        if (j < n - 1 && a[j] < a[j + 1]) {
            j++;
        }

        if (v >= a[j]) {
            break;
        }

        a[k] = a[j];
        k = j;
    }

    a[k] = v;
}

```

```

void heap_sort(float* a, int n)
{
    int i;
    float t;

    for (i = n / 2 - 1; i >= 0; i--) {
        downheap(a, n, i);
    }

    while (n > 1) {
        t = a[0];
        a[0] = a[n - 1];
        a[n - 1] = t;

        n--;
        downheap(a, n, 0);
    }
}

```

`downheap()`은 heap sort에서 부모 노드가 자식보다 작을 때 아래로 내려 보내면서 max heap 구조를 유지하는 함수이다. 먼저 $v = a[k]$ 로 현재 부모 값을 저장하고, $j = 2 * k + 1$ 을 이용해 왼쪽 자식의 위치를 구한다. 오른쪽 자식이 존재하고 오른쪽 자식이 더 크면 $j++$ 로 더 큰 자식을 선택한다. 그다음 부모값 v 가 선택된 자식 $a[j]$ 보다 크거나 같으면 이미 heap 조건을 만족하므로 멈추고, 그렇지 않으면 큰 자식 값을 부모 자리로 올린 뒤 $k = j$ 로 위치를 아래로 이동한다. 마지막에 내려간 위치에 원래 부모값 v 를 넣어 heap 구조를 다시 맞춘다.

`heap_sort()`는 배열을 오름차순으로 정렬하는 함수이다. 먼저 `for (i = n / 2 - 1; i >= 0; i--)` 반복문에서 마지막 부모 노드부터 루트까지 `downheap()`을 적용해 배열 전체를 max heap으로 만든다. max heap이 되면 가장 큰 값이 항상 $a[0]$ 에 있으므로, `while (n > 1)` 반복문에서 $a[0]$ 과 맨 끝 값 $a[n - 1]$ 을 바꿔 가장 큰 값을 배열 끝으로 보낸다. 그 후 정렬이 끝난 마지막 원소를 제외하기 위해 $n--$ 을 하고, 다시 `downheap(a, n, 0)`을 호출해 남은 부분을 max heap으로 만든다. 이 과정을 반복하면 큰 값부터 뒤로 차례대로 고정되면서 최종적으로 배열이 오름차순으로 정렬된다.

```

int main() {
    int height = 44, width = 45;
    int window_size = 3;

    float* input = (float*)malloc(sizeof(float) * height * width);
    float* output = (float*)malloc(sizeof(float) * height * width);

    if (input == NULL || output == NULL) {
        printf("Memory allocation failed!\n");
        return -1;
    }

    char cwd[1024];
    _getcwd(cwd, sizeof(cwd));
    printf("Current working directory: %s\n", cwd);

    read_file(input, height, width, "noisy_data.txt");
    printf("Data loaded successfully.\n");

    median_filtering(input, output, height, width, window_size);
    printf("Filtering completed.\n");

    FILE* out_fp = fopen("filtered_result2.txt", "w");

    if (out_fp == NULL) {
        printf("Output file open failed!\n");
        exit(1);
    }

    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {
            fprintf(out_fp, "%.0f\t", output[y * width + x]);
        }
        fprintf(out_fp, "\n");
    }

    fclose(out_fp);

    printf("Filtered result saved to filtered_result.txt\n");

    free(input);
    free(output);

    return 0;
}

```

이 main() 함수는 프로그램 전체 실행 흐름을 담당한다. 먼저 데이터 크기를 height = 44, width = 45로 정하고, median filter의 크기를 window_size = 3으로 설정한 뒤, 원본 데이터를 저장할 input 배열과 필터링 결과를 저장할 output 배열을 동적 할당한다. 메모리 할당에 실패하면 오류 메시지를 출력하고 종료하며, 정상적으로 할당되면 현재 실행 경로를 출력한 뒤 read_file()을 호출하여 noisy_data.txt의 숫자 데이터를 input에 읽어온다. 필터링이 끝나면 filtered_result.txt 파일을 쓰기 모드로 열고, output 배열에 저장된 44×45 결과값을 행과 열 형태로 파일에 저장한다. 마지막으로 파일을 닫고 저장 완료 메시지를 출력한 뒤, 동적 할당했던 input과 output 메모리를 해제하고 프로그램을 종료한다.

3. 코드 시뮬레이션

- height = 44, width = 45, window_size = 3으로 설정
- input과 output 배열을 동적 할당
- read_file()로 noisy_data.txt의 44×45 데이터를 input에 저장
- median_filtering()을 실행하여 각 위치 주변 3×3 값을 temp에 저장
- temp 값을 heap_sort()로 정렬한 뒤 가운데 값을 median으로 선택
- 선택된 median 값을 output 배열에 저장
- 이 과정을 전체 44×45 데이터에 대해 반복
- output 결과를 filtered_result.txt 파일에 저장
- input, output 메모리를 해제하고 프로그램 종료

4. 후기

이번 코드를 작성하면서 텍스트 파일에서 44×45 데이터를 읽어 배열에 저장하고, 각 위치 주변의 3×3 값을 이용해 median filtering을 수행하는 전체 흐름을 이해할 수 있었다. 처음에는 파일 경로나 출력 결과 때문에 데이터가 제대로 읽히지 않는 것처럼 보였지만, 현재 실행 폴더를 확인하고 결과를 txt 파일로 저장하면서 원본 데이터와 필터링 결과를 비교할 수 있었다. 또한 주변값을 임시 배열에 저장하고 heap sort로 정렬한 뒤 중앙값을 선택하는 과정을 통해 정렬 알고리즘이 실제 이미지 노이즈 제거에 어떻게 활용되는지 확인할 수 있었다. 특히 console 출력만으로는 결과를 확인하기 어렵기 때문에 파일로 저장해 검토하는 방식이 더 적절하다는 점도 알게 되었다.